

6.Graph và Tree

6. Graph và Tree khác nhau như thế nào? Khi nào dùng Graph, khi nào dùng Tree?

Tiêu chí	Tree (Cây)	Graph (Đồ thị)
Bản chất	Là một dạng đặc biệt của Đồ thị.	Là cấu trúc dữ liệu bao gồm các Đỉnh (Vertex) và Cạnh (Edge).
Cấu trúc	Phân cấp (Hierarchical) theo mô hình quan hệ cha - con.	Mạng lưới (Network) không có quy tắc phân cấp cố định.
Node Gốc (Root)	Chỉ có duy nhất 1 node gốc.	Không có khái niệm node gốc (các node bình đẳng với nhau).
Chu trình (Cycle)	Tuyệt đối không có chu trình (không thể đi từ 1 node và quay lại chính nó mà không đi lùi).	Có thể có hoặc không có chu trình.
Đường đi (Path)	Chỉ có duy nhất 1 đường đi giữa 2 node bất kỳ.	Có thể có nhiều đường đi giữa 2 node.
Số lượng cạnh	Nếu có N node, sẽ có chính xác N-1 cạnh.	Số cạnh linh hoạt, phụ thuộc vào số lượng kết nối.
Duyệt (Traversal)	Pre-order, In-order, Post-order, Level-order.	BFS (Breadth-First Search), DFS (Depth-First Search).

Ứng dụng của TREE:

Thư mục máy tính, CSDL, Autocomplete.

Ứng dụng của Graph:

Social Networks(Face, Zalo, Insta), Navigation Maps

1. Queue (Hàng đợi) <-> Thuật toán BFS (Breadth-First Search - Duyệt theo chiều rộng)

Queue hoạt động theo nguyên tắc **FIFO (First In, First Out - Vào trước ra trước)**. Khi áp dụng vào Tree hoặc Graph, nó tạo ra hiệu ứng duyệt theo từng lớp, giống như vết dầu loang.

Ứng dụng trên Tree: **heapq, Level-order Traversal** (Duyệt theo mức).

Ứng dụng trên Graph: Dùng để tìm đường đi ngắn nhất (Shortest Path) trong đồ thị không có trọng số. Ví dụ: Tìm số trạm trung chuyển ít nhất để đi từ ga A đến ga B.

Bảng Tóm Tắt Mối Liên Hệ

Tiêu chí	Đi với Queue	Đi với Stack (hoặc Đệ quy)
Thuật toán cốt lõi	BFS (Breadth-First Search)	DFS (Depth-First Search)
Chiến lược duyệt	Lan tỏa đều ra xung quanh (như sóng nước).	Đi thẳng một mạch đến ngõ cụt rồi mới quay lui (Backtracking).
Trên Tree	Level-order (Duyệt theo từng tầng).	Pre/In/Post-order.
Trên Graph	Tìm đường đi ngắn nhất (ít cạnh nhất).	Kiểm tra chu trình, duyệt mê cung.
Bản chất ghi nhớ	Nhớ các node kề để lát nữa duyệt.	Nhớ các node để tí nữa quay lui lại.

7. DFS và BFS

7. Giải thích khái niệm và so sánh DFS và BFS. Hãy suy nghĩ và đưa ra một ví dụ trong hệ thống thương mại điện tử trường hợp nào dùng DFS, trường hợp nào là BFS?

DFS: Duyệt theo chiều sâu, đi đến tận cùng nhánh đang duyệt rồi mới quay lại. (LIFO)

BFS: Duyệt theo chiều rộng, từ nhánh “cha” duyệt qua từng nhánh “con” rồi mới duyệt xuống “cháu”. (FIFO)

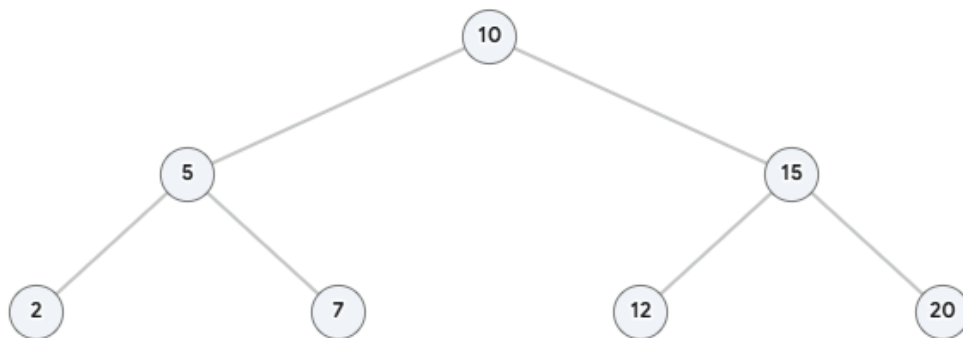
Ví dụ:

BFS: Gợi ý sản phẩm liên quan, VD: Laptop “cha”, duyệt và gợi ý các “con” như túi chống sốc, dán màn hình, bảo hiểm,...

DFS: Dùng kế thừa thuộc tính cho Danh mục khi Update Sale.

“Điện tử” khuyến mãi 20%, các mục con “Điện thoại”, “laptop” đến ngách đều được update lần lượt theo mỗi nhánh đến gốc(giảm bộ nhớ lưu trữ).

8. Binary Search Tree (BST)



1. Minh họa hàm `bst_search(value)`

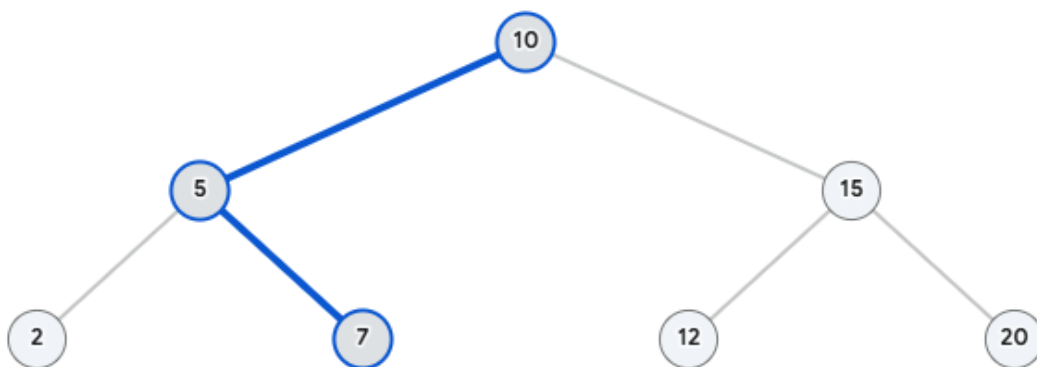
Mục tiêu: Tìm xem giá trị `value` có tồn tại trong cây hay không.

Ví dụ: Tìm `value = 7` trong cây.

- **Bước 1:** Bắt đầu ở node gốc (10).
- **Bước 2:** So sánh: $7 < 10$ -> Rẽ nhánh **Trái**.
- **Bước 3:** Gặp node (5). So sánh: $7 > 5$ -> Rẽ nhánh **Phải**.
- **Bước 4:** Gặp node (7). So sánh: $7 == 7$ -> **Đã tìm thấy!** (Trả về node 7).

Ví dụ: Tìm `value = 8` trong cây.

- Làm tương tự đến khi đứng ở node (7). So sánh $8 > 7$ -> Cần rẽ nhánh **Phải**.
- Tuy nhiên, node (7) là node lá, bên phải của nó là rỗng. -> Dừng lại và trả về `"None"` (Không tìm thấy).

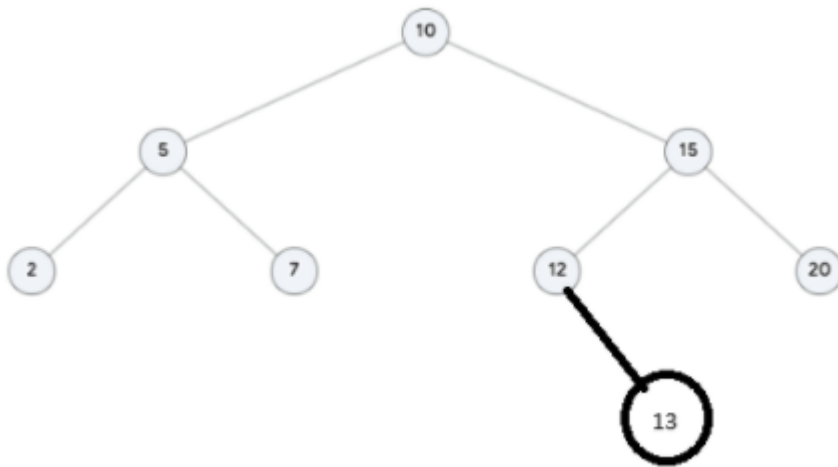


2. Minh họa hàm **bst_insert(value)**

Mục tiêu: Đưa một giá trị mới vào đúng vị trí để cây vẫn giữ nguyên tính chất BST.

Ví dụ: Chèn **value = 13** vào cây.

- **Bước 1:** Bắt đầu ở gốc (10).
- **Bước 2:** $13 > 10$ -> Đi sang nhánh **Phải**.
- **Bước 3:** Gặp node (15). So sánh $13 < 15$ -> Đi sang nhánh **Trái**.
- **Bước 4:** Gặp node (12). So sánh $13 > 12$ -> Ta tạo một node mới mang giá trị (13) và nối nó làm con phải của (12)



9 Adjacency List hoặc Adjacency Matrix.

9. Đồ thị có thể biểu diễn bằng Adjacency List hoặc Adjacency Matrix.

Adjacency Matrix:

- Ma trận $N \times N$
- $O[i][j] = 1$ nếu có cạnh

Adjacency List:

- Mỗi node lưu danh sách hàng xóm

Đề bài: Hệ thống có 10,000 kho, mỗi kho kết nối trung bình 5 kho khác → nên chọn cách nào? Vì sao?

Gợi ý:

Với 10,000 node: Matrix cần bao nhiêu ô nhớ? Với mỗi node chỉ có ~5 cạnh: Đây là graph dày hay thưa?

Đây là Graph thưa với tổng 10.000 kho, chỉ cần dùng Adjacency List để lưu danh sách hàng xóm.

10.Chu trình trong Graph

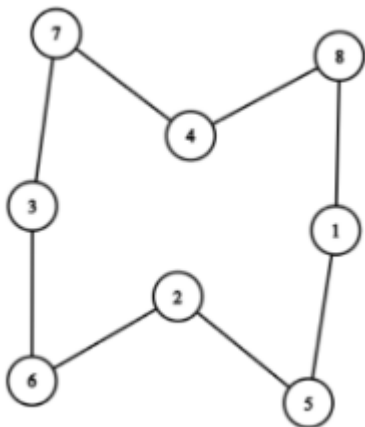
10. Chu trình trong Graph:

Chu trình (Cycle) trong đồ thị (graph) là một đường đi khép kín, trong đó đỉnh đầu và đỉnh cuối trùng nhau, và các cạnh/đỉnh nằm giữa được đi qua không quá một lần.

Ví dụ: Xuất phát từ (1) -> quay lại được về (1) khi qua tất cả các cạnh đúng 1 lần.

Đề bài: Làm sao để kiểm tra một đồ thị có chu trình hay không?

Gợi ý: Sử dụng DFS, cần lưu thêm gì để phát hiện cycle?



Khi duyệt đồ thị, duy trì 2 thông tin:

1. **Mảng visited (Đã thăm):** Để theo dõi những đỉnh nào đã được duyệt qua.
2. **Biến parent (Đỉnh cha):** Lưu vết đỉnh liền trước mà từ đó ta đi đến đỉnh hiện tại.

Bắt đầu từ 1 lưu parent là 0, visited 1

8: parent là 1, visited là 8

4: parent là 8, visited là 4

7: parent là 4, visited là 7

3: parent là 7, visited là 3

6: parent là 3, visited là 6

2: parent là 6, visited là 2

5: parent là 2, visited là 5

Ở đỉnh 5 node tiếp theo là 1 == visited đã lưu and != parent hiện tại.

Kết luận: Đồ thị có chu trình

